

Bridge Day 19 Challenge Battery

Cumulative practice after Exam 2 Question 20

Thursday, July 30, 2026 • 34 problems • STUDENT PACKET • successor 07302026__v10

Name	Section
------	---------

Purpose and construct boundary

This is a cumulative problem bank built from the reasoning used in Exam 2 Question 20: indexing, first-seen order, parallel state, nested search, counting, mutation, and tie handling. It introduces no functions. All programs are complete top-level programs.

Trace items	Read the program, keep a written state ledger, and write exact printed output without a computer.
Build items	Write one complete top-level program. Change only the supplied data assignments when testing.
Difficulty	SLICK turns on one subtle mechanic. SCOPE requires tracking many pieces of state.
Exactness	Brackets, commas, spaces, quotes, labels, and capitalization count.

Route. Do not try to finish all 34 problems. Work until the problems stop being easy, then slow down, keep a written ledger, and explain the state changes.

Tier	Problems	Demand
1 - warm-up traces	6	one hidden mechanic each
2 - slick traces	12	subtle behavior distinguishes near-looking programs
3 - scope traces	6	many moving pieces; a ledger is required
4 - build programs	10	complete top-level programs with restrictions

Tier 1 - Warm-up traces

Short programs. Each hides one mechanic.

A1. Index or value

SLICK

```
values = [3, 1, 4, 1, 5]
count = 0

for v in values:
    for j in range(len(values)):
        if v == j:
            break
        else:
            count = count + 1

print(count)
```

Output: _____

End A1

A2. What the loop variable keeps

SLICK

```
for row in range(3):
    for col in range(4):
        if row + col == 3:
            break
    print(row, col)
```

Output: _____

End A2

A3. Which side the piece lands on

SLICK

```
tags = ["a", "b", "c", "d"]
out = ""

for i in range(len(tags)):
    out = tags[i] + out

print(out)
```

Output: _____

End A3

A4. Letter i of word i

SLICK

```
words = ["logic", "bool", "float", "graph"]
secret = ""

for i in range(len(words)):
    secret = secret + words[i][i]

print(secret)
```

Output: _____

End A4

A5. One short of the end

SLICK

```
temps = [95, 70, 80, 99]
hot = 0

for i in range(len(temps) - 1):
    if temps[i] > 90:
        hot = hot + temps[i]

print(hot)
```

Output: _____

End A5

A6. Past the limit

SLICK

```
value = 1
steps = 0

while value < 40:
    if value % 3 == 0:
        value = value * 2
    else:
        value = value + 5
    steps = steps + 1

print(value, steps)
```

Output: _____

End A6

Tier 2 - Slick traces

Near-looking programs can behave differently. Name the state change that matters.

B1. Reading what you already wrote

SLICK

```
data = [1, 2, 3, 4, 5]

for i in range(1, len(data)):
    data[i] = data[i] + data[i - 1]

print(data)
```

Output: _____

End B1

B2. The same loop, backwards

SLICK

```
data = [1, 2, 3, 4, 5]

for i in range(len(data) - 1, 0, -1):
    data[i] = data[i] + data[i - 1]

print(data)
```

Output: _____

End B2

B3. A flag that never clears

SLICK

```
values = [2, 9, 4, 1]
seen_big = False
labels = []

for v in values:
    if v > 5:
        seen_big = True
    if seen_big:
        labels.append("after")
    else:
        labels.append("before")

print(labels)
```

Output: _____

End B3

B4. Where the counter is born

SLICK

```
total = 0

for station in range(3):
    checks = 0
    for slot in range(station + 1):
        checks = checks + 1
        total = total + 1
    print(station, checks)

print(total)
```

Output: _____

End B4

B5. Digits from the back

SLICK

```
number = 4073
digits = []
total = 0

while number > 0:
    digits.append(number % 10)
    total = total + number % 10
    number = number // 10

print(digits, total)
```

Output: _____

End B5

B6. Swap in the wrong order

SLICK

```
pair = [7, 2]
pair[0] = pair[1]
pair[1] = pair[0]
print(pair)

other = [7, 2]
hold = other[0]
other[0] = other[1]
other[1] = hold
print(other)
```

Output: _____

End B6

B7. Reversing too far

SLICK

```
items = [1, 2, 3, 4]

for i in range(len(items)):
    hold = items[i]
    items[i] = items[len(items) - 1 - i]
    items[len(items) - 1 - i] = hold

print(items)
```

Output: _____

End B7

B8. Stopping the wrong loop

SLICK

```
found = 0

for a in range(3):
    for b in range(3):
        if a + b == 2:
            found = found + 1
            break
    if a == 1:
        continue
    found = found + 10

print(found)
```

Output: _____

End B8

B9. Every other letter, from where

SLICK

```
word = "engineering"
print(word[0:9:2])
print(word[1:9:3])
print(word[-4:])
print(word[-1] + word[0])
```

Output: _____

End B9

B10. Two lists, one offset

SLICK

```
left = [10, 20, 30, 40]
gaps = []

for i in range(len(left) - 1):
    gaps.append(left[i + 1] - left[i])

print(gaps, len(gaps))
```

Output: _____

End B10

B11. The condition that shrinks

SLICK

```
queue = [3, 1]
seen = 0

while seen < len(queue):
    if queue[seen] > 2:
        queue.append(1)
        seen = seen + 1

print(queue, seen)
```

Output: _____

End B11

B12. Counting matches against itself

SLICK

```
nums = [4, 7, 4]
pairs = 0

for i in range(len(nums)):
    for j in range(len(nums)):
        if nums[i] == nums[j]:
            pairs = pairs + 1

print(pairs)
```

Output: _____

End B12

Tier 3 - Scope traces

Track the state on paper. Do not try to hold it all in your head.

C1. Run lengths

SCOPE

```
codes = ["a", "a", "b", "b", "b", "c", "a", "a"]
runs = []
lengths = []
current = codes[0]
count = 1

for i in range(1, len(codes)):
    if codes[i] == current:
        count = count + 1
    else:
        runs.append(current)
        lengths.append(count)
        current = codes[i]
        count = 1

runs.append(current)
lengths.append(count)

print(runs)
print(lengths)
print(len(runs))
```

Output: _____

End C1

C2. Merge without sorting

SCOPE

```
a = [1, 4, 9]
b = [2, 3, 10, 12]
merged = []
i = 0
j = 0

while i < len(a) and j < len(b):
    if a[i] <= b[j]:
        merged.append(a[i])
        i = i + 1
    else:
        merged.append(b[j])
        j = j + 1

while i < len(a):
    merged.append(a[i])
    i = i + 1

while j < len(b):
    merged.append(b[j])
    j = j + 1

print(merged, i, j)
```

Output: _____

End C2

C3. One bubble pass

SCOPE

```
values = [5, 1, 4, 2, 8]
swaps = 0

for i in range(len(values) - 1):
    if values[i] > values[i + 1]:
        hold = values[i]
        values[i] = values[i + 1]
        values[i + 1] = hold
        swaps = swaps + 1

print(values, swaps)
```

Output: _____

End C3

C4. Skip, stop, and keep score

SCOPE

```
grid = [4, 0, 7, 3, 0, 9]
total = 0
kept = []

for i in range(len(grid)):
    if grid[i] == 0:
        continue
    if total > 12:
        break
    total = total + grid[i]
    kept.append(grid[i])

print(kept, total, i)
```

Output: _____

End C4

C5. Longest climb

SCOPE

```
heights = [1, 3, 5, 2, 4, 6, 8, 3]
best = 1
current = 1

for i in range(1, len(heights)):
    if heights[i] > heights[i - 1]:
        current = current + 1
        if current > best:
            best = current
    else:
        current = 1

print(best, current)
```

Output: _____

End C5

C6. Vowels per word, and the winner

SCOPE

```
words = ["rhythm", "audio", "queue", "sky"]
counts = []

for w in words:
    c = 0
    for ch in w:
        if ch in "aeiou":
            c = c + 1
    counts.append(c)

best = 0
for i in range(len(counts)):
    if counts[i] > counts[best]:
        best = i

print(counts)
print(words[best], counts[best])
```

Output: _____

End C6

Tier 4 - Build complete programs

Use only top-level code. Run every supplied dataset without changing the algorithm.

D1. Compress the runs

SCOPE

Write a complete program. A list named `codes` is supplied. Print `Runs`: followed by the letters naming each consecutive run, then `Lengths`: followed by the matching run lengths.

Restrictions: Top-level code only. Use loops, indexing, comparison, and `append`. No dictionaries, functions, or shortcuts.

Test datasets and expected output

Data assignments	Exact output
<code>codes = ['a', 'a', 'b', 'c', 'c', 'c']</code>	<code>Runs: ['a', 'b', 'c']</code> <code>Lengths: [2, 1, 3]</code>
<code>codes = ['x']</code>	<code>Runs: ['x']</code> <code>Lengths: [1]</code>
<code>codes = []</code>	<code>Runs: []</code> <code>Lengths: []</code>
<code>codes = ['a', 'b', 'a', 'b']</code>	<code>Runs: ['a', 'b', 'a', 'b']</code> <code>Lengths: [1, 1, 1, 1]</code>

Program workspace

End D1

D2. Second largest, no shortcuts

SLICK

Write a complete program. A list named `values` is supplied. Print `Second`: followed by the second-largest different value, or -1 when no second different value exists.

Restrictions: Top-level code only. No `max()`, `sort()`, or `sorted()`. Use your own comparisons.

Test datasets and expected output

Data assignments	Exact output
<code>values = [4, 9, 7, 9]</code>	<code>Second: 7</code>
<code>values = [5, 5, 5]</code>	<code>Second: -1</code>
<code>values = [3]</code>	<code>Second: -1</code>
<code>values = [-4, -9]</code>	<code>Second: -9</code>

Program workspace

End D2

D3. Keep the first of each

SCOPE

Write a complete program. A list named `values` is supplied. Print `Kept:` followed by a new list with duplicates removed while preserving first-seen order.

Restrictions: Top-level code only. No sets, dictionaries, comprehensions, `count()`, or `index()`. Search what has already been kept.

Test datasets and expected output

Data assignments	Exact output
<code>values = [3, 1, 3, 2, 1]</code>	<code>Kept: [3, 1, 2]</code>
<code>values = []</code>	<code>Kept: []</code>
<code>values = [7, 7, 7]</code>	<code>Kept: [7]</code>
<code>values = [1, 2, 3]</code>	<code>Kept: [1, 2, 3]</code>

Program workspace

End D3

D4. Merge two sorted lists

SCOPE

Write a complete program. Increasing lists named `left` and `right` are supplied. Print `Merged:` followed by one increasing list containing every value.

Restrictions: Top-level code only. No `sort()` or `sorted()`. Advance a position in each list.

Test datasets and expected output

Data assignments	Exact output
<code>left = [1, 4, 9]; right = [2, 3, 10]</code>	<code>Merged: [1, 2, 3, 4, 9, 10]</code>
<code>left = []; right = [1, 2]</code>	<code>Merged: [1, 2]</code>
<code>left = [5]; right = []</code>	<code>Merged: [5]</code>
<code>left = [1, 1]; right = [1, 1]</code>	<code>Merged: [1, 1, 1, 1]</code>

Program workspace

End D4

D5. Digital root

SLICK

Write a complete program. A positive whole number named `number` is supplied. Repeatedly add its digits until one digit remains. Print `Root:` and `Rounds:` on separate lines.

Restrictions: Top-level code only. Use arithmetic and loops. Do not convert the number to a string.

Test datasets and expected output

Data assignments	Exact output
<code>number = 4073</code>	<code>Root: 5</code> <code>Rounds: 2</code>
<code>number = 9</code>	<code>Root: 9</code> <code>Rounds: 0</code>
<code>number = 99999</code>	<code>Root: 9</code> <code>Rounds: 2</code>
<code>number = 10</code>	<code>Root: 1</code> <code>Rounds: 1</code>

Program workspace

End D5

D6. Longest common start

SCOPE

Write a complete program. A list named `words` is supplied. Print `Common:` followed by the longest text every word starts with. Print an empty string when there is none.

Restrictions: Top-level code only. Use indexing and loops. Do not use slicing to skip comparisons.

Test datasets and expected output

Data assignments	Exact output
<code>words = ['engine', 'engineer', 'engrave']</code>	<code>Common: eng</code>
<code>words = ['abc', 'xyz']</code>	<code>Common:</code>
<code>words = ['same', 'same']</code>	<code>Common: same</code>
<code>words = []</code>	<code>Common:</code>

Program workspace

End D6

D7. Rotate right

SCOPE

Write a complete program. A list named `values` and whole number named `k` are supplied. Move every value `k` positions right with wraparound, then print Rotated: followed by the changed list.

Restrictions: Top-level code only. No slicing. Work out where each value must land.

Test datasets and expected output

Data assignments	Exact output
<code>values = [1, 2, 3, 4, 5]; k = 2</code>	Rotated: [4, 5, 1, 2, 3]
<code>values = [1, 2, 3]; k = 3</code>	Rotated: [1, 2, 3]
<code>values = [1, 2, 3]; k = 4</code>	Rotated: [3, 1, 2]
<code>values = []; k = 2</code>	Rotated: []

Program workspace

End D7

D8. Most frequent, earliest tie

SCOPE

Write a complete program. A list named `values` is supplied. Print Most: and Count: on separate lines. If counts tie, keep the value that appears first. For an empty list, print -1 and 0.

Restrictions: Top-level code only. No dictionaries, sets, `count()`, `max()`, or comprehensions.

Test datasets and expected output

Data assignments	Exact output
<code>values = [4, 7, 4, 7, 4]</code>	Most: 4 Count: 3
<code>values = [1, 2, 2, 1]</code>	Most: 1 Count: 2
<code>values = [9]</code>	Most: 9 Count: 1
<code>values = []</code>	Most: -1 Count: 0

Program workspace

End D8

D9. Balance with refused changes

SCOPE

Write a complete program. A starting number named balance and list named changes are supplied. Apply changes in order; refuse any change that would make the balance negative. Print Balance: and Refused:.

Restrictions: Top-level code only. Use one pass and no second list.

Test datasets and expected output

Data assignments	Exact output
balance = 100; changes = [-30, -90, 50, -20]	Balance: 100 Refused: 1
balance = 0; changes = [-1, 1]	Balance: 1 Refused: 1
balance = 10; changes = []	Balance: 10 Refused: 0
balance = 5; changes = [-5, -1]	Balance: 0 Refused: 1

Program workspace

End D9

D10. Every target position

SCOPE

Write a complete program. A list named values and a target are supplied. Print Positions: followed by every matching index, then Gap: followed by last index minus first index. Use -1 when absent.

Restrictions: Top-level code only. No index() method. One pass is enough.

Test datasets and expected output

Data assignments	Exact output
values = [4, 7, 4, 1, 4]; target = 4	Positions: [0, 2, 4] Gap: 4
values = [1, 2]; target = 9	Positions: [] Gap: -1
values = [5]; target = 5	Positions: [0] Gap: 0
values = [3, 3]; target = 3	Positions: [0, 1] Gap: 1

Program workspace

End D10